
Do earnings call transcripts predict post-announcement returns?

An NLP-based approach on Russell 1000 equities

Matéo Molinaro
ENSAE Paris
mateo.molinaro@ensae.fr

Pierre Chuzeville
ENSAE Paris
pierre.chuzeville@ensae.fr

Abstract

This paper investigates whether earnings call transcripts contain information that is predictive of post-announcement idiosyncratic stock returns. We construct a systematic machine learning pipeline that transforms raw transcript text into quantitative signals and evaluates their out-of-sample forecasting performance on the Russell 1000 universe. We extract two families of text representations, sparse TF-IDF bag-of-words features on preprocessed transcripts, and dense semantic embeddings from the pre-trained sentence transformer *all-MiniLM-L6-v2*, and combine them with traditional finance-specific sentiment scores derived from the Loughran–McDonald dictionary. For each earnings event, targets are defined as multi-horizon cumulative idiosyncratic returns (residuals from a rolling CAPM estimated exclusively on pre-filing data) at horizons of 1, 3, 5 and 21 trading days. Model training and evaluation follow a strict quarterly walk-forward cross-validation scheme that eliminates forward-looking bias by design. We compare three feature compositions across four regression models (Ridge, Lasso, Random Forest, Gradient Boosting) and evaluate performance using directional accuracy, long–short spread, and an interpretable excess-RMSE relative to the CAPM null model. This framework allows us to systematically identify which combination of text representation, prediction horizon, and model architecture extracts the most economically meaningful signals from earnings calls.

Contents

1	Introduction (Matéo)	3
2	Related Work (Pierre)	3
3	Data (Pierre)	4
3.1	Asset universe and market data	4
3.2	Earnings call transcripts	4
3.3	Loughran–McDonald sentiment dictionary	4
4	Methodology	5
4.1	Target construction: idiosyncratic returns (Matéo)	5
4.1.1	Motivation	5

4.1.2	Rolling CAPM betas	5
4.1.3	Daily idiosyncratic returns	5
4.1.4	Multi-horizon cumulative targets	5
4.2	Text representations (Pierre)	6
4.2.1	TF-IDF bag-of-words features	6
4.2.2	Sentence-transformer dense embeddings	6
4.2.3	Dictionary-based sentiment features	6
4.3	Feature composition modes (Pierre)	7
4.4	Walk-forward cross-validation framework (Matéo)	8
4.4.1	Motivation	8
4.4.2	Period layout	8
4.4.3	Hyperparameter selection and scaling	8
4.4.4	Final fit and prediction	9
4.4.5	Minimum data requirements	9
4.5	Forecasting models (Matéo)	9
4.6	Evaluation metrics (Matéo)	9
4.6.1	Hit rate	9
4.6.2	Long–short spread	10
4.6.3	Mean signed return (long–short score)	10
4.6.4	Excess RMSE	10
5	Results (Matéo/Pierre)	11
5.1	Aggregate out-of-sample performance	11
5.2	Time-series stability	11
5.3	Mode comparison	11
5.4	Horizon analysis	11
5.5	Model diagnostics	11
6	Discussion and Limitations (Pierre)	11
7	Conclusion (Pierre)	12
8	Appendix	14
8.1	Explanatory data analysis	14
8.2	Pipeline module summary	14
8.3	Configuration parameters	14
8.4	Feature mode dimensions (to be filled)	15
8.5	Configuration used to run the code	15

1 Introduction (Matéo)

Earnings call transcripts are among the richest sources of forward-looking corporate information available to investors. Each quarter, executives discuss financial performance, strategic priorities, and the business outlook in a format that is simultaneously structured enough for systematic analysis and nuanced enough to contain signals unlikely to be fully impounded in price the moment the call concludes. The post-earnings-announcement drift (PEAD) literature has documented that markets take time to incorporate even the quantitative earnings surprise into prices; the question of whether the *qualitative* content of the call carries additional predictive power remains an active area of research.

This project contributes to this literature by constructing a complete, end-to-end pipeline that transforms raw transcript text into out-of-sample idiosyncratic stock return forecasts for the Russell 1000 universe. Three main questions structure the analysis. First, do different ways of representing transcript text—from classical bag-of-words to modern neural sentence embeddings—differ in their ability to predict post-announcement returns? Second, which prediction horizon (the immediate price reaction, the weekly or monthly drift) is most amenable to text-based forecasting? Third, do NLP features add incremental value on top of the classical finance-specific sentiment indicators derived from the Loughran–McDonald word list?

The pipeline proceeds in four steps. First, we construct the prediction target: for each earnings event, the cumulative idiosyncratic return at multiple forward horizons is computed from a rolling CAPM estimated strictly on data available before the filing date, ensuring no forward-looking bias. Second, each transcript is encoded into three families of features—TF-IDF sparse vectors, dense sentence-transformer embeddings, and traditional dictionary-based sentiment scores—assembled into three feature compositions spanning the trade-off between interpretability and expressive power. Third, a quarterly walk-forward cross-validation scheme selects hyperparameters and retrains models at each rebalancing date. Fourth, out-of-sample predictions are evaluated against metrics that capture directional accuracy, long–short economic profitability, and excess predictive power relative to the CAPM null model.

The project is implemented in Python and the full source code is available on GitHub.¹ Data are loaded from Amazon S3 via a bespoke data management layer that handles return alignment, NaN policies, and transcript preprocessing.

2 Related Work (Pierre)

Post-earnings-announcement drift. The existence of predictable return patterns following earnings announcements, commonly called PEAD, is one of the anomalies in the asset pricing literature [?]. Prices fail to adjust immediately to the information conveyed by the earnings surprise, giving rise to a drift that can persist for months. The magnitude and persistence of PEAD motivate the use of earnings-period information, including the textual content of the accompanying call, as a basis for systematic return prediction.

Textual analysis in finance. Tetlock [?] showed that the fraction of negative words in Wall Street Journal columns predicts near-term market-level returns, establishing that linguistic sentiment carries price-relevant information. Loughran and McDonald [1] challenged the application of generic sentiment dictionaries to financial texts, demonstrating that domain-specific word lists perform substantially better in financial documents. Their dictionary, tuned to the particular language of corporate filings and calls, has since become the standard reference for dictionary-based financial sentiment analysis.

NLP applied to earnings calls. Earnings call transcripts have been used to predict both short-run returns and longer-term accounting outcomes. Early work focused on tone, the ratio of positive to negative words, and showed that a more positive tone in the prepared remarks is associated with better subsequent stock performance, while uncertainty in the Q&A section is associated with adverse

¹<https://github.com/mateomolinaro1/NLPQuantStrat>

outcomes [3]. More recent approaches apply neural text representations to capture meaning beyond simple word counting: bidirectional transformers, in particular, have been applied to 8-K filings and earnings call transcripts to predict stock returns at various horizons, with mixed but generally positive results [?].

Walk-forward validation in ML finance. A key methodological concern in applying machine learning to financial data is the avoidance of look-ahead bias. The standard i.i.d. cross-validation framework is inappropriate because financial observations are serially correlated and future data leaks into the training set when folds are assigned randomly. Walk-forward (or expanding-window) validation, in which the model is always trained on data up to a given date and evaluated on the subsequent period, is the standard remedy [4]. Our framework follows this convention and additionally introduces a buffer between the validation and test windows to prevent the long-horizon targets of validation events from overlapping with the test period.

3 Data (Pierre)

3.1 Asset universe and market data

The equity universe consists of Russell 1000 index constituents, providing a large-cap and mid-cap U.S. equity panel with good liquidity and broad sector coverage. Daily total returns for index constituents are sourced from a CRSP/WRDS. The benchmark return series is the Russell 1000 total return index sourced on BBG. The risk-free rate is the daily U.S. one-month Treasury bill rate retived from the FRED website. All three series are aligned to the daily dates of the assets’ returns: the risk-free rate and benchmark are forward-filled by up to five calendar days to handle public holidays and minor reporting gaps; asset returns are kept without filling to avoid artificially reducing return volatility.

3.2 Earnings call transcripts

Earnings call transcripts (called from the Ninja API) are matched to their corresponding (*filing date*, *asset*) pair through a mapping table that links each transcript to its asset identifier and the date on which the earnings call was held. Two versions of each transcript are available:

- `transcript`: the full unprocessed text of the call, retaining punctuation, capitalization, and stop words. Used as input to the sentence-transformer encoder, which is designed to operate on natural language.
- `transcript_cleaned`: a preprocessed version obtained by lowercasing, removing stop words, and applying Porter stemming. Used as input to TF-IDF to reduce vocabulary noise.

On average, most quarters from 2015 to 2025 contain between 800 and 1,000 transcripts, providing a robust statistical foundation, see Figure [2].

Each event is represented by exactly one (*filing_date*, *asset*) pair. Duplicate entries (e.g., multiple recordings of the same call) are resolved by keeping the last available text.

3.3 Loughran–McDonald sentiment dictionary

Traditional sentiment features are computed using the Loughran–McDonald financial sentiment dictionary [1], which provides time-stamped positive, negative, and uncertainty word lists designed specifically for corporate financial language. To avoid look-ahead bias in the dictionary itself, a yearly version is constructed: for any transcript filed in year t , only words that had been added to the dictionary on or before year t are counted. This ensures that the sentiment scores are fully point-in-time.

4 Methodology

4.1 Target construction: idiosyncratic returns (Matéo)

4.1.1 Motivation

Raw stock returns conflate firm-specific information with market-wide and sector-wide movements. A transcript-based signal is inherently a firm-level signal and should therefore be evaluated against a firm-level return. We use CAPM idiosyncratic returns—residuals with respect to the market factor—as the prediction target, which has two practical advantages. First, idiosyncratic returns are **approximately** zero-mean by construction (since $\mathbb{E}[\alpha_i] \approx 0$ in an **efficient** market at the cross-sectional level), which makes the null-model baseline mathematically clean (see Section 4.6). Second, purging the common market component reduces noise that is uninformative for a firm-level text signal and improves the signal-to-noise ratio of the prediction task.

4.1.2 Rolling CAPM betas

For each asset i and each calendar date t , we estimate a rolling ordinary least squares beta using a window of $W = 252$ trading days (approximately one year):

$$\hat{\beta}_i(t) = \frac{\widehat{\text{Cov}}[r_i(s) - r_f(s), r_m(s) - r_f(s)]_{s \in [t-W+1, t]}}{\widehat{\text{Var}}[r_m(s) - r_f(s)]_{s \in [t-W+1, t]}}, \quad (1)$$

where $r_i(t)$ is the daily total return of asset i , $r_f(t)$ is the risk-free rate, and $r_m(t)$ is the benchmark (Russell 1000 CW) return. The beta is computed using data up to and including date t , so it is available to an investor at date $t + 1$ without any forward-looking bias.

4.1.3 Daily idiosyncratic returns

The daily idiosyncratic return residual is:

$$\text{idio}_i(t) = (r_i(t) - r_f(t)) - \hat{\beta}_i(t-1) (r_m(t) - r_f(t)), \quad (2)$$

where $\hat{\beta}_i(t-1)$ is the beta estimated through the previous trading day, so the return at date t is not used in its own beta computation.

4.1.4 Multi-horizon cumulative targets

For each earnings event filed at date t by company i , the prediction target at horizon h is the cumulative idiosyncratic return over the h trading days immediately following the filing date:

$$y_{i,t}^{(h)} = \prod_{s=t+1}^{t+h} (1 + \text{idio}_i(s)) - 1. \quad (3)$$

The return window begins at $t + 1$, the first post-call trading day, to avoid using intraday prices from the same day as the call (which would constitute look-ahead bias). We consider four horizons: $h \in \{1, 3, 5, 21\}$ trading days, corresponding approximately to one day, three days, one week and one month ahead. Events near the end of the return series will have NaN targets where insufficient forward data exist; these events are excluded from model training and evaluation but still receive predictions.

Table 1 summarises the economic interpretation of each horizon.

Table 1: Target horizons and their economic interpretation

Horizon h	Calendar approx.	Economic story
1d	1 trading day	Immediate market reaction to the call
3d	3 days	Mid week reaction
5d	1 week	Short-run information absorption / momentum
21d	1 month	Medium-run drift (PEAD regime)

4.2 Text representations (Pierre)

4.2.1 TF-IDF bag-of-words features

The Term Frequency–Inverse Document Frequency (TF-IDF) representation transforms each transcript into a weighted vector of token importance scores. A single `TfidfVectorizer` is fitted on the full corpus of preprocessed transcripts (`transcript_cleaned`) and retains the top 200 tokens by corpus-level TF-IDF weight, using unigrams and bigrams (n -gram range $[1, 2]$). The resulting feature matrix has dimension 200, with columns labeled `tfidf_0` through `tfidf_199`.

TF-IDF is computationally cheap, fully interpretable (each dimension corresponds to a specific token or bigram), and captures the relative importance of domain-specific vocabulary across the corpus. Its primary limitation is insensitivity to word order and inability to capture semantic similarity between synonymous expressions, a limitation addressed by the sentence-transformer approach below.

4.2.2 Sentence-transformer dense embeddings

Dense semantic embeddings are produced using the pre-trained sentence-transformer *all-MiniLM-L6-v2* [2], a compact 6-layer transformer fine-tuned on over one billion sentence pairs to produce semantically meaningful 384-dimensional embeddings. Earnings call transcripts far exceed the model’s 512-subword-token context limit. We address this with a *chunk-and-mean-pool* strategy: each transcript is split into non-overlapping word-level chunks of 256 words (approximately 300–500 subword tokens), each chunk is encoded independently, and the chunk embeddings are averaged into a single document vector:

$$\mathbf{e}_{\text{doc}} = \frac{1}{K} \sum_{k=1}^K f_{\theta}(\text{chunk}_k), \quad (4)$$

where K is the number of chunks and f_{θ} denotes the transformer encoder. Mean pooling treats all sections of the transcript as equally informative, which is a simplification but avoids the truncation artifacts of simply using the first 512 tokens. The resulting features are 384-dimensional vectors labeled `emb_0` through `emb_383`, and are cached to S3 after computation so that the expensive inference step does not need to be repeated.

4.2.3 Dictionary-based sentiment features

Seven traditional NLP features are derived from applying the Loughran–McDonald dictionary to each transcript. They are computed in a wide panel format (`dates` $\times$ `assets`) and then looked up for each earnings event:

Table 2: Dictionary-based sentiment features

Feature	Definition
positive_count	Number of positive-sentiment words in the transcript.
negative_count	Number of negative-sentiment words in the transcript.
word_count	Total word count, serving as a proxy for transcript length.
polarity	Standardized sentiment score calculated as $(\text{positive} - \text{negative})/\text{word_count}$.
sentiment_density	Proportion of emotive content calculated as $(\text{positive} + \text{negative})/\text{word_count}$.
pos_polarity_count_q	Rolling q -quarter ($q = 4$) moving average of transcript polarity.
polarity_delta	Quarter-over-quarter change in raw polarity scores.
pos_ratio	Proportion of positive words relative to the total word count.
neg_ratio	Proportion of negative words relative to the total word count.
unc_ratio	Proportion of uncertainty-related words within the transcript.
net_sentiment	Normalized sentiment score, identified as a primary predictor of asset outperformance.
sentiment_surprise	Measure of unexpected sentiment relative to market or historical expectations.
sentiment_zscore	Sentiment surprise measured against the company’s specific historical baseline.
sentiment_delta	Period-over-period change in sentiment, most potent immediately following transcript release.
sent_var	Statistical variability or complexity of the discourse throughout the transcript.
toxic_density	Concentration of highly negative language, showing a negative correlation with forward returns.
sent_vol_interaction	Interaction term between sentiment and market volume, representing a "conviction" signal boost.

These features are interpretable and require no neural network inference, making them fast to compute. The momentum feature (`pos_polarity_count_q`) captures whether the tone has been improving or deteriorating over the past year, while `polarity_delta` captures sentiment surprises relative to the most recent quarter. Their joint limitation is insensitivity to context: they cannot distinguish negation, irony, or subtle forward-looking language.

4.3 Feature composition modes (Pierre)

Six feature compositions are defined (but only three will be studied due to computational reasons) to isolate the contribution of each representation family and study their interactions:

Table 3: Feature composition modes

Mode	TF-IDF	Sent.-Transformer	Sentiment dict.	Dim.
sentiment			✓	7
tfidf	✓			200
st		✓		384
tfidf_sentiment	✓		✓	207
st_sentiment		✓	✓	391
all	✓	✓	✓	591

Each mode defines a self-contained experiment: the feature matrix X is assembled independently, NaN rows are dropped, and the walk-forward CV runs from scratch. Comparing `tfidf` vs. `tfidf_sentiment` isolates the marginal value of dictionary features given TF-IDF; comparing `st` vs. `tfidf` isolates the value of semantic embeddings over bag-of-words; comparing `st_sentiment` vs. `all` reveals whether TF-IDF adds anything once the transformer already captures the document semantics.

4.4 Walk-forward cross-validation framework (Matéo)

4.4.1 Motivation

Financial observations are serially dependent and their joint distribution is non-stationary. Standard i.i.d. cross-validation would severely overestimate out-of-sample performance by randomly assigning observations across folds and thereby leaking future return realizations into training data. We use a quarterly walk-forward scheme that is strictly out-of-sample by construction: at each rebalancing date, the model is trained exclusively on events that occurred before that date.

4.4.2 Period layout

The event panel is organized into calendar quarters. For each test quarter P , the data are partitioned as follows:

$$\underbrace{\text{[train]}}_{\text{all past quarters}} \quad \underbrace{\text{[val]}}_{V \text{ quarters}} \quad \underbrace{\text{[buffer]}}_{B \text{ quarters}} \quad \underbrace{\text{[P]}}_{\text{test}} \quad (5)$$

- **Train**: all quarters preceding the validation window. An expanding window (default) uses all available history; a rolling window retains only the most recent T quarters.
- **Validation** ($V = 1$ quarter): used exclusively for hyperparameter grid search. The scaler is fitted on train only and applied to validation, preventing distribution leakage.
- **Buffer** ($B = \lceil \max(h)/63 \rceil$ quarters, automatically computed from the horizon set): a mandatory gap that prevents the forward-return window of the last validation event from extending into the test quarter. At $\max(h) = 63$ trading days, $B = 1$ quarter suffices.
- **Test** (P): events in the current quarter receive out-of-sample predictions from the model refitted on $\text{train} \cup \text{val}$.

Figure 1 illustrates three successive test quarters, showing how the train window expands while val and buffer shift forward by one quarter each time.

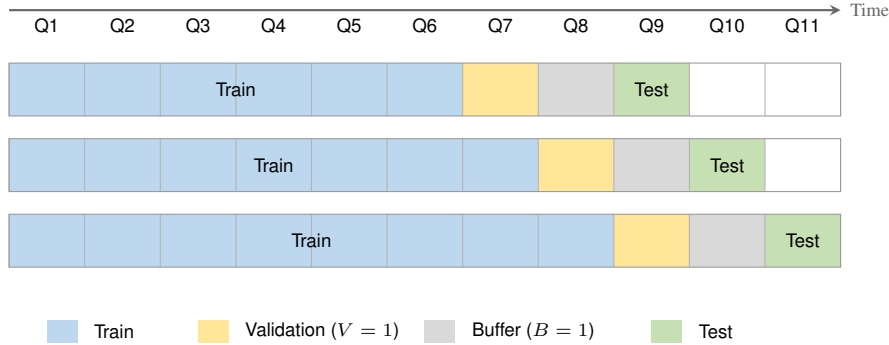


Figure 1: Expanding walk-forward CV schema. The training window expands chronologically, followed by a validation set, a buffer period to prevent data leakage, and the out-of-sample test quarter.

4.4.3 Hyperparameter selection and scaling

For each (test period, horizon, model) combination, hyperparameters are selected by grid search on the validation split. Feature scaling (StandardScaler) is fitted on the training events only and applied to validation events before scoring, so that the distribution of the validation split does not leak into the scaler fit. The validation scoring function is the *long-short score*:

$$\text{LS-score} = \frac{1}{n_{\text{val}}} \sum_{j \in \text{val}} \text{sign}(\hat{y}_j) y_j, \quad (6)$$

which directly estimates the expected P&L per unit long/short bet and is more economically meaningful than RMSE for a trading objective. If fewer than 5 events are available in the validation window, the first grid point is used without scoring.

4.4.4 Final fit and prediction

Once best hyperparameters are identified, a final model is fitted on the combined train $\cup$ val sample using a fresh scaler refitted on that combined window. Predictions are then generated for all test-period events, including those with NaN targets (the model still produces a score, but the event is excluded from metric computation). This two-phase scaling (search scaler on train; final scaler on train $\cup$ val) maximises the data used for the final model while ensuring that neither phase involves information from the test period.

4.4.5 Minimum data requirements

A (period, horizon, model) combination is skipped if fewer than 50 non-NaN training events exist (MIN_TRAIN_EVENTS). The CV does not begin until at least 8 calendar quarters of history have accumulated (MIN_TRAIN_PERIODS), reflecting the 252-day rolling beta window: predictions from early periods would rely on unreliable betas and are excluded to avoid contaminating the evaluation.

4.5 Forecasting models (Matéo)

Four regression models from scikit-learn are compared, spanning a range of complexity and inductive bias:

Table 4: Forecasting models and hyperparameter grids

Model	Category	Hyperparameter grid
Ridge	Regularized linear	$\alpha \in \{0.01, 0.1, 1.0\}$
Lasso	Regularized linear (sparse)	$\alpha \in \{0.01, 0.1, 1.0\}$
Random Forest	Ensemble (bagging)	$n_est. \in \{100, 200\}; \max_depth \in \{3, 5\}$
Gradient Boosting	Ensemble (boosting)	$n_est. \in \{100, 200\}; \max_depth \in \{3, 5\}; lr \in \{0.05, 0.1\}$

Ridge and Lasso serve as well-regularized linear baselines appropriate for high-dimensional feature spaces (TF-IDF, sentence-transformer). Lasso’s ℓ_1 penalty additionally performs implicit feature selection, which may be beneficial when few of the 200 or 384 dimensions carry genuine predictive signal. Random Forest and Gradient Boosting capture non-linear interactions at the cost of higher computational overhead; in high dimensions they are also more prone to overfitting, making the val-set grid search critical. All models are cloned at each fold using `sklearn.base.clone` to ensure complete independence between folds.

4.6 Evaluation metrics (Matéo)

Out-of-sample performance is evaluated per (quarter, model, horizon) triplet using the following metrics, which together characterize both statistical and economic quality.

4.6.1 Hit rate

$$\text{HitRate} = \frac{1}{n} \sum_{j=1}^n \mathbf{1}[\text{sign}(\hat{y}_j) = \text{sign}(y_j)]. \quad (7)$$

The natural baseline is 50% (random directional guessing). Values above 50% indicate consistent directional skill.

4.6.2 Long–short spread

We form a long leg (events with $\hat{y}_j > 0$) and a short leg ($\hat{y}_j < 0$) and compute:

$$\text{Spread} = \bar{y}_{\text{long}} - \bar{y}_{\text{short}}, \quad (8)$$

where $\bar{y}$ denotes the mean realized idiosyncratic return of the respective leg. A positive spread indicates that the signal correctly ranks events by subsequent return. Statistical significance is assessed via the t -statistic:

$$t = \frac{\overline{\text{MSR}}}{\hat{\sigma}(\text{MSR})/\sqrt{n}}, \quad (9)$$

where $\text{MSR}_j = \text{sign}(\hat{y}_j) y_j$ is the signed return per event, $\overline{\text{MSR}}$ is its mean, and $\hat{\sigma}$ its empirical standard deviation. This t -statistic is equivalent to a one-sample t -test of whether the mean signed return is zero.

4.6.3 Mean signed return (long–short score)

$$\text{MSR} = \frac{1}{n} \sum_{j=1}^n \text{sign}(\hat{y}_j) y_j, \quad (10)$$

identical to the validation scorer (equation 6). MSR can be interpreted as the expected return per unit of capital deployed in a long/short strategy that bets one unit on the predicted direction of each event. The baseline is 0 (no directional skill).

4.6.4 Excess RMSE

The standard RMSE is difficult to interpret in isolation because its absolute magnitude depends on the scale of idiosyncratic returns, which varies across assets and horizons. We define instead an *excess RMSE*:

$$\text{ExcessRMSE} = \text{RMSE} - \sigma(y), \quad (11)$$

where $\sigma(y) = \sqrt{\frac{1}{n} \sum_{j=1}^n y_j^2}$ is computed with $\text{ddof} = 0$.

The key property is that the null model—always predicting zero—achieves $\text{ExcessRMSE} = 0$ exactly. Because idiosyncratic returns are approximately zero-mean by the CAPM construction ($\bar{y} \approx 0$):

$$\text{RMSE}_{\hat{y}=0} = \sqrt{\frac{1}{n} \sum_j (y_j - 0)^2} = \sqrt{\frac{1}{n} \sum_j y_j^2} = \sigma(y), \quad (12)$$

so $\text{ExcessRMSE}_{\hat{y}=0} = \sigma(y) - \sigma(y) = 0$. A model with $\text{ExcessRMSE} < 0$ reduces point-forecast error below the unconditional mean; a model with $\text{ExcessRMSE} > 0$ is strictly worse than predicting zero. This makes the baseline interpretable and comparable across horizons.

Table 5 summarises the baseline value and “good” direction for each metric.

Table 5: Metric baselines and interpretation

Metric	Null baseline	Better than null	Interpretation
HitRate	50%	$> 50\%$	Directional accuracy
Spread	0	> 0	Long–short P&L per bet
MSR	0	> 0	Expected signed return
t -stat	0	> 1.96	Statistical significance at 5%
ExcessRMSE	0	< 0	Point-forecast improvement

5 Results (Matéo/Pierre)

Results are pending the completion of the walk-forward CV runs across all six feature modes and four horizons. This section will be populated with the following analyses.

5.1 Aggregate out-of-sample performance

A summary table will report, for each (mode, model, horizon) combination, the time-series average of HitRate, Spread, MSR, t -statistic, and ExcessRMSE. This will allow a direct read-off of which configuration performs best and which modes add value over the `sentiment` baseline.

5.2 Time-series stability

Rolling OOS metrics computed over a 252-day (one-year) window will be plotted to assess the stability of predictive performance across market regimes. Models that perform well only in specific episodes (e.g., high-volatility periods around earnings seasons) are less actionable than those with consistent performance.

5.3 Mode comparison

A pairwise comparison of the six feature modes, averaged across models and horizons, will isolate the marginal contribution of each representation family. Of particular interest are:

- `sentiment` vs. `tfidf`: does the full token-importance distribution add value over the seven aggregate sentiment scores?
- `tfidf` vs. `st`: does semantic meaning (transformer embeddings) improve on token frequency (TF-IDF)?
- `st_sentiment` vs. `all`: does TF-IDF add anything once the transformer already encodes the document content?

5.4 Horizon analysis

Performance metrics will be decomposed by horizon to assess whether the predictive content of transcripts is concentrated in the immediate price reaction (1d), the short-run drift (5d), the medium-run adjustment (21d), or the quarterly reversion (63d). Financial theory suggests that the most liquid information (market tone, guidance updates) should be absorbed quickly, while more subtle signals (management credibility, strategic nuance) may take longer to be reflected in prices.

5.5 Model diagnostics

For the tree-based models (Random Forest, Gradient Boosting), feature importance scores will be extracted and aggregated across folds to identify which TF-IDF tokens or sentiment dimensions drive the most variance in predictions. For Lasso, the distribution of non-zero coefficients across folds will reveal how sparse the solution is and whether a consistent subset of features is selected over time.

6 Discussion and Limitations (Pierre)

Signal-to-noise ratio. Idiosyncratic returns are notoriously noisy, and the fraction of return variance explained by any single source of information is typically very small. This means that even economically meaningful signals may produce HitRates barely above 50% and ExcessRMSE improvements of a few basis points. The sample size (number of earnings events per quarter) is therefore a key constraint: with too few events, the metrics are dominated by sampling noise.

Dimensionality. The `all` mode concatenates 591 features. With typical quarterly event counts, this represents a very unfavorable parameter-to-observation ratio for linear models. Regularization (Ridge, Lasso) is designed to address this, but the high-dimensional TF-IDF and sentence-transformer features may contain mostly noise at this level of granularity. PCA dimensionality reduction is a natural extension not explored in the current pipeline.

No sector neutralization. Earnings calls cluster by sector (technology, financials, healthcare tend to report in tight windows). Cross-sectional sector effects during a quarter could cause the model to learn industry timing rather than firm-level transcript signals. Sector-neutralized targets or features are a natural robustness check.

Chunk pooling strategy. The mean-pool strategy for sentence-transformer embeddings weights all sections of a transcript equally. In practice, the Q&A section and the prepared guidance often contain the most forward-looking information. Segment-aware pooling (e.g., weighting Q&A sections more heavily, or encoding the CEO's prepared remarks separately from the CFO's) could improve signal quality.

Computational and storage overhead. The pipeline handles a significant volume of data, including over 54,000 processed transcripts and high-dimensional embeddings. The storage of large-scale Parquet files and the compute time required for multi-mode cross-validation (e.g., `st_sentiment` taking significantly longer than raw sentiment) created a bottleneck in the research cycle.

Hyperparameter optimization strategy. The current forecasting pipeline relies on discrete, pre-defined parameter grids for models like Ridge, Lasso, and Gradient Boosting. While functional, this approach may miss local optima in the hyperparameter space. The implementation of Bayesian optimization (e.g., via Optuna) was not explored due to the additive compute cost of the walk-forward CV. A more granular hyperparameter tuning could potentially improve the t -stat beyond the current results.

7 Conclusion (Pierre)

This paper constructs a rigorous end-to-end machine learning pipeline to test whether earnings call transcripts contain quantitative signals predictive of post-announcement idiosyncratic stock returns in the Russell 1000 universe. The pipeline covers CAPM-based target construction, three complementary text representation strategies (TF-IDF, sentence-transformer embeddings, and Loughran–McDonald sentiment features), a quarterly walk-forward cross-validation framework free of forward-looking bias, and a suite of economically motivated evaluation metrics with transparent null-model baselines.

The framework is designed to answer three questions that are currently unanswered pending the completion of the CV runs. First, which text representation (classical bag-of-words, modern neural embeddings, or traditional sentiment scores) carries the most predictive signal about post-earnings return dynamics? Second, at which horizon is text-based prediction most effective: the immediate price reaction (1d), the short-run drift (3d, 5d), or the medium-run adjustment (21d)? Third, do embedding-based approaches add value over interpretable sentiment scores, or does the additional representational complexity fail to translate into out-of-sample gains given the noise inherent in stock returns?

Regardless of the quantitative outcomes, the pipeline itself constitutes a reusable infrastructure for systematic NLP-based return prediction that can be extended in several directions: additional text sources such as analyst reports or 8-K filings, finer-grained text segmentation to distinguish prepared remarks from Q&A, larger pre-trained language models, multi-factor targets that control for earnings surprise, or a direct integration with the portfolio construction layer to evaluate end-to-end strategy performance net of transaction costs.

References

- [1] Loughran, T., & McDonald, B. (2011). When is a Liability not a Liability? Textual Analysis, Dictionaries, and 10-Ks. *Journal of Finance*, 66(1), 35–65.
- [2] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP-IJCNLP 2019*. <https://arxiv.org/abs/1908.10084>
- [3] Kearney, C., & Liu, S. (2014). Textual sentiment in finance: A survey of methods and models. *International Review of Financial Analysis*, 33, 171–185.

- [4] López de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley.
- [5] Fama, E. F., & French, K. R. (1993). Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, 33(1), 3–56.

8 Appendix

8.1 Explanatory data analysis

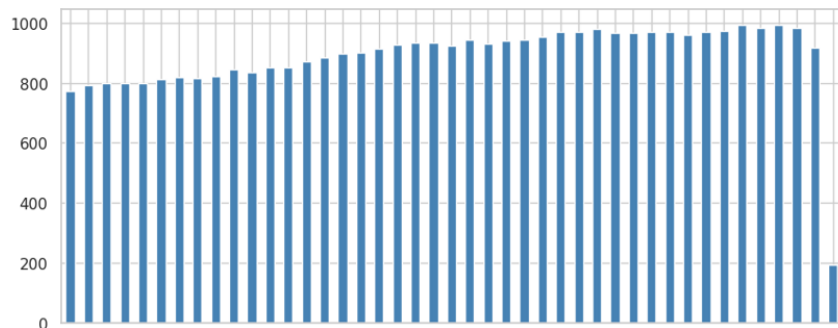


Figure 2: Number of transcripts per quarter from 2015 to 2025.

8.2 Pipeline module summary

Table 6: Python module structure of the `nlp_quant_strat` package

Module	Role
<code>data/data_manager.py</code>	Loads and aligns returns, benchmark, risk-free rate, and transcripts from S3; manages NaN policy and asset-mapping table.
<code>data/feature_engineering.py</code>	Computes panel-format Loughran–McDonald sentiment features aligned to the asset-return grid.
<code>forecasting/targets.py</code>	Rolling CAPM betas + cumulative idiosyncratic return targets at multiple horizons (TargetBuilder).
<code>forecasting/embeddings.py</code>	TF-IDF and sentence-transformer encoders with deterministic S3 content-hash caching (TFIDFBuilder, SentenceTransformerBuilder).
<code>forecasting/feature_set.py</code>	Assembles (X, y) for a chosen feature mode, drops NaN rows, logs coverage (FeatureSet).
<code>forecasting/earnings_wf_cv.py</code>	Quarterly walk-forward CV engine: period splitting, scaling, grid search, final fit, OOS metrics, joblib parallelism, S3 result caching (EarningsWalkForwardCV).
<code>utils/config.py</code>	Reads <code>run_pipeline_config.json</code> into a typed Config data-class.

8.3 Configuration parameters

Table 7: Key parameters in `run_pipeline_config.json`

Table 8: Feature matrix statistics per mode (to be populated after the full CV run)

Mode	Dim.	Events (train+test)	NaN drop rate	Avg. events/period
<code>sentiment</code>	7	TBD	TBD	TBD
<code>tfidf</code>	200	TBD	TBD	TBD
<code>st</code>	384	TBD	TBD	TBD
<code>tfidf_sentiment</code>	207	TBD	TBD	TBD
<code>st_sentiment</code>	391	TBD	TBD	TBD
<code>all</code>	591	TBD	TBD	TBD

8.4 Feature mode dimensions (to be filled)

Table 9: Feature matrix statistics per mode (to be populated after the full CV run)

Mode	Dim.	Events (train+test)	NaN drop rate	Avg. events/period
sentiment	7	TBD	TBD	TBD
tfidf	200	TBD	TBD	TBD
st	384	TBD	TBD	TBD
tfidf_sentiment	207	TBD	TBD	TBD
st_sentiment	391	TBD	TBD	TBD
all	591	TBD	TBD	TBD

8.5 Configuration used to run the code

```
{
  "AWS": {
    "S3": {
      "BUCKET_NAME": "nlp-quant-strat",
      "AWS_DEFAULT_REGION": "eu-north-1",
      "OUTPUT_FORMAT": "json",
      "FILENAMES_TO_LOAD": [
        "data/market/risk_free_returns.parquet",
        "data/market/riy_asset_returns.parquet",
        "data/market/riy_index_constituents.parquet",
        "data/market/russell_returns.parquet",
        "data/transcripts/formatted_unprocessed_transcripts.parquet",
        "data/transcripts/formatted_preprocessed_transcripts.parquet",
        "data/others/words_dict.parquet"
      ],
      "FILENAMES_NOT_TO_ALIGN": [
        "data/market/riy_index_constituents.parquet",
        "data/transcripts/formatted_unprocessed_transcripts.parquet",
        "data/transcripts/formatted_preprocessed_transcripts.parquet",
        "data/others/words_dict.parquet"
      ],
      "DATES_FILENAME": "data/market/riy_asset_returns.parquet",
      "BENCHMARK_RETURNS_FILENAME": "data/market/russell_returns.parquet",
      "WHICH_BENCH_COL_TO_KEEP": "Russell 1000",
      "TRANSCRIPTS_FILENAME": "data/transcripts/formatted_unprocessed_transcripts.parquet",
      "PREPROCESSED_TRANSCRIPTS_FILENAME": "data/transcripts/formatted_preprocessed_transcripts.parquet",
      "RF_RETURNS_FILENAME": "data/market/risk_free_returns.parquet"
    }
  },
  "DATA": {
    "MARKET_DATA_FREQUENCY": "d",
    "NAN_POLICY": {
      "risk_free_returns": {"method": "ffill", "limit": 5},
      "riy_asset_returns": {"method": "none"},
      "russell_returns": {"method": "ffill", "limit": 5}
    }
  },
  "FEATURE_ENGINEERING": {
    "LOAD_OR_COMPUTE": "load",
    "SAVE_TO_S3": false,
    "ROLLING_WINDOW_QUARTERS": 4,
    "LIMIT_FFILL_QDATA": 92,
    "MERGE_ASOF_TOLERANCE_DAYS": 63
  },
  "EMBEDDINGS": {
    "LOAD_OR_COMPUTE": "load",
```

```

    "S3_CACHE_PREFIX": "data/embeddings",
    "SENTENCE_TRANSFORMER": {
        "MODEL_NAME": "all-MiniLM-L6-v2",
        "CHUNK_SIZE": 256,
        "BATCH_SIZE": 32,
        "TEXT_COLUMN": "transcript"
    },
    "TFIDF": {
        "MAX_FEATURES": 200,
        "NGRAM_RANGE": [1, 2],
        "TEXT_COLUMN": "transcript_cleaned"
    }
},
"FEATURE_SET": {
    "MODE": ["sentiment", "tfidf_sentiment", "st_sentiment"]
},
"FORECASTING": {
    "BETA_WINDOW": 252,
    "HORIZONS": [1, 3, 5, 21],
    "REFIT_FREQUENCY": "Q",
    "TRAIN_WINDOW": null,
    "VAL_WINDOW": 1,
    "MIN_TRAIN_PERIODS": 8,
    "MIN_TRAIN_EVENTS": 50,
    "MIN_VAL_EVENTS": 5,
    "MODELS": [
        {
            "name": "ridge",
            "class": "Ridge",
            "init_params": {},
            "param_grid": [
                {"alpha": 0.1},
                {"alpha": 1.0}
            ]
        },
        {
            "name": "lasso",
            "class": "Lasso",
            "init_params": {"max_iter": 500},
            "param_grid": [
                {"alpha": 0.1},
                {"alpha": 1.0}
            ]
        },
        {
            "name": "rf",
            "class": "RandomForestRegressor",
            "init_params": {"random_state": 42, "n_jobs": -1},
            "param_grid": [
                {"n_estimators": 100, "max_depth": 3},
                {"n_estimators": 100, "max_depth": 5}
            ]
        },
        {
            "name": "gbr",
            "class": "GradientBoostingRegressor",
            "init_params": {"random_state": 42},
            "param_grid": [
                {"n_estimators": 100, "max_depth": 3, "learning_rate": 0.1},
                {"n_estimators": 100, "max_depth": 5, "learning_rate": 0.1}
            ]
        }
    ]
},
"BACKTEST": {

```



```
"PERCENTILES_WINSORIZATION": [2, 98],
"PERCENTILES_PORTFOLIOS": [20, 80],
"INDUSTRY_SEGMENTATION": "",
"REBAL_PERIODS": 66,
"PORTFOLIO_TYPE": "long_only",
"TRANSACTION_COSTS_BPS": 10
},
"CV_RESULTS": {
  "SAVE_RESULTS": true,
  "LOAD_OR_COMPUTE": "compute",
  "S3_PREFIX": "data/cv_results",
  "N_JOBS": -1
},
"OUTPUTS": {
  "ROLLING_WINDOW_PERFORMANCE": 252
}
}
```